



Panchip Microelectronics Co., Ltd.

PAN3029/3060 Series SDK User Guide

Current version: 0.6

Release date: 2024.04

Shanghai Panqi Microelectronics Co., Ltd.

Address: Room 302, Building D, No. 666, Shengxia Road, Zhangjiang Hi-Tech Park,
Shanghai Tel: 021-50802371

Website: <http://www.panchip.com>

Documentation

Due to version upgrades or other reasons, the content of this document will be updated from time to time. Unless otherwise agreed, the content of this document is only for guidance, and all statements, information and suggestions in this document do not constitute any express or implied warranty.

Trademark

Panqi is a trademark of Panqi Microelectronics Co., Ltd. Other names mentioned in this document are trademarks/registered trademarks of their respective owners

Disclaimer

All or part of the products, services or features described in this document may not be within the scope of your purchase or use. Unless otherwise agreed in the contract, Panqi Microelectronics Co., Ltd. makes no express or implied statement or warranty regarding the contents of this document.

Revision History

Version	Revision date	Description
V0.1	2023.06	Initial version
V0.2	2023.08	1.Modify the operation process after using single packet mode 2.Added interface for obtaining current channel energy detection
V0.3	2023.09	1.Update transmit power table 2.Update MAPM interface
V0.4	2023.12	1.Update transmit power table 1-23 Gear
V0.5	2023.12	1.Update SDK Migration Instructions
V0.6	2024.04	Modify function interface and chip description

Table of contents

1 Overview.....	1
2 SDK Usage.....	2
2.1 Initialization.....	2
2.1.1 Interface	2
2.1.2 Implementation.....	2
2.1.3 Usage	2
2.2 Parameters Configuration	2
2.2.1 Interface	2
2.2.2 Implementation.....	2
2.2.3 Usage	3
2.3 Transmitting.....	3
2.3.1 Interface	3
2.3.2 Implementation.....	3
2.3.3 Usage	4
2.4 Receiving.....	4
2.4.1 Interface	4
2.4.2 Implementation.....	4
2.4.3 Usage	4
3 SDK interface description	5
3.1 rf_flag interface.....	5
3.2 rf_init.....	6
3.3 rf_sleep_wakeup	6
3.4 rf_deepsleep	7
3.5 rf_sleep	7
3.6 rf_get_tx_time	7
3.7 rf_set_mode	8
3.8 rf_get_mode	8
3.9 rf_set_tx_mode	9
3.10 rf_set_rx_mode	9
3.11 rf_set_rx_single_timeout	10
3.12 rf_get_snr	10
3.13 rf_get_rssi	11
3.14 rf_get_channel_rssi	11
3.15 rf_get_irq	12
3.16 rf_clr_irq	12
3.17 rf_set_refresh	13
3.18 rf_set_preamble	13
3.19 rf_cad_on	14
3.20 rf_cad_off	14

3.21 rf_set_syncword	15
3.22 rf_get_syncword	15
3.23 rf_early interrupt interface	16
3.24 rf_set_mapm_rx_on	16
3.25 rf_set_mapm_rx_off	16
3.26 rf_set_mapm_para	17
3.27 rf_receive	17
3.28 rf_enter_continuous_rx	18
3.29 rf_enter_single_timeout_rx	18
3.30 rf_enter_single_rx	19
3.31 rf_single_tx_data	19
3.32 rf_enter_continuous_tx/rf_continuous_tx_send_data	20
3.33 rf_set_agc	20
3.34 rf_set_dcdc_mode	21
3.35 rf_set_ldr	21
3.36 rf_intelligent search interface	22
3.37 get_chirp_time	22
3.38 check_cad_rx_inactive	23
4 SDK Porting	24
5 PAN3029/3060 Series Status Diagram	25
6 Application Notes	27
6.1 Low Power Sleep	27
6.2 CAD Function	27
6.3 Sensitivity and RSSI	27
6.4 DCDC Function	28
6.5 EFUSE Function	28
6.6 RF Parameters Description	30
6.7 4-wire SPI and FIFO	31
7 Appendix	33
7.1 Transmitting power table	33

1 Overview

This document mainly introduces the SDK interface functions of the PAN3029/3060 series chips, including the chip initialization process and the data transmitting and receiving process.

SDK related files include "pan_rf.c", "pan_rf.h", "pan_port.c", "pan_port.h", and "pan_param.h".

2 SDK Usage

This chapter describes in detail the basic process of using the SDK, including the most basic initialization, parameter configuration, and data transmission and reception.

2.1 Initialization

2.1.1 Interface

```
rf_init();
```

2.1.2 Implementation

1. Wake up the chip from deep sleep state to standby3 state
2. Load FT configuration
3. Initialize register configuration
4. AGC Configuration
5. Initialize chip TX_IO/RX_IO

2.1.3 Usage

Called when the chip is powered on for the first time or wakes up from deepsleep.

2.2 Parameters Configuration

2.2.1 Interface

```
rf_set_default_para();
```

2.2.2 Implementation of functions

1. Set frequency, Code_Rate, bandwidth, spreading factor, transmit power and CRC

```
rf_set_freq(DEFAULT_FREQ);  
rf_set_code_rate(DEFAULT_CR);  
rf_set_bw(DEFAULT_BW);  
rf_set_sf(DEFAULT_SF);  
rf_set_crc(CRC_ON);  
rf_set_tx_power(DEFAULT_PWR);
```

2.2.3 Usage

After the chip is initialized, the interface function is called in the standby3 state.

2.3 Transmitting

2.3.1 Interface

```
rf_enter_continuous_tx();  
rf_continuous_tx_send_data();  
rf_single_tx_data();
```

2.3.2 Implementation

1. Switch to STB3 state

```
rf_set_mode(RF_MODE_STB3)
```

2. Open PA

```
rf_set_ldo_pa_on
```

3. Open TX_IO

```
rf_port.set_tx
```

4. Set the transmission mode

```
rf_set_tx_mode
```

5. Enter TX state and send data

```
rf_send_packet
```

6. Get the transmission time of this packet data (optional)

```
rf_get_tx_time
```

2.3.3 Usage

After the chip is initialized and the configuration parameters are completed, the interface function is called. After the interface function is called and the chip is sent, the IRQ pin will be set high and an interrupt will be generated.

2.4 Receiving

2.4.1 Interface

```
rf_enter_continuous_rx();  
rf_enter_single_timeout_rx();  
rf_enter_single_rx();
```

2.4.2 Implementation

1. Switch to STB3 state

```
rf_set_mode(RF_MODE_STB3)
```

2. Open RX_IO

```
rf_port.set_rx
```

2. Set RX mode

```
rf_set_rx_mode
```

4. Enter receiving state

```
rf_set_mode(RF_MODE_RX)
```

2.4.3 Usage

After the chip is initialized and the configuration parameters are completed, the interface function is called. After receiving, the IRQ pin will be set high. You can use polling or interrupt to determine whether the data packet is received. After receiving it, call the packet receiving function to take the data packet from the chip.

3 SDK interface description

This chapter describes in detail the user interface functions provided by the SDK.

Most parameter configurations need to be performed after the chip is initialized (in standby3 state).

3.1 rf_flag interface

Functions

```
int rf_get_recv_flag(void)
void rf_set_recv_flag(int status)
int rf_get_transmit_flag(void)
void rf_set_transmit_flag(int status)
```

Purpose

After the chip is set to send or receive, an IRQ interrupt is generated, and the event is passed to the user through the flag. The user can use the polling method to determine the chip's sending or receiving results.

Parameter

```
Current event status
RADIO_FLAG_IDLE
RADIO_FLAG_TXDONE
RADIO_FLAG_RXDONE
RADIO_FLAG_RXTIMEOUT
RADIO_FLAG_RXERR
RADIO_FLAG_PLHDRXDONE
RADIO_FLAG_MAPM
```

3.2 rf_init

Function

```
RF_Err_t rf_init(void)
```

Purpose

Initialize the chip, including wake-up, load FT configuration, initialize register configuration, AGC configuration.

Called when the chip is powered on for the first time or wakes up from deepsleep.

Return Value

FAIL Operation failed

OK Operation successful

3.3 rf_sleep_wakeup

Function

```
RF_Err_t rf_sleep_wakeup(void)
```

Purpose

Set the chip to wake up from sleep state.

When the chip is in sleep state, it is used to wake-up the chip.

When the chip is in sleep state, the register value of the chip will not be reset. After waking up from sleep, the parameter configuration before sleep is retained.

Return Value

FAIL Operation failed

OK Operation successful

3.4 rf_deepsleep

Function

```
RF_Err_t rf_deepsleep(void)
```

Purpose

Switch the chip from standby3 state to deep sleep state to reduce power consumption.

Return Value

FAIL Operation failed

OK Operation successful

3.5 rf_sleep

Function

```
RF_Err_t rf_sleep(void)
```

Purpose

Switch the chip from standby3 state to sleep state to reduce power consumption.

Return Value

FAIL Operation failed

OK Operation successful

3.6 rf_get_tx_time

Function

```
uint32_t rf_get_tx_time(uint8_t size)
```

Purpose

Get the time required to transmit this data packet. The usage of this function is demonstrated in rf_single_tx_data().

Parameter

Size: The length of the data packet to transmit

Return Value

The time required to transmit this data packet, in us.

3.7 rf_set_mode

Function

```
RF_Err_t rf_set_mode(uint8_t mode)
```

Purpose

Set the chip operation mode, including deepsleep/sleep/standby1/standby2/standby3/tx/rx. Parameter configuration is usually performed in standby3 state.

Parameter

Mode Chip operation mode

RF_MODE_DEEP_SLEEP

RF_MODE_SLEEP

RF_MODE_STB1

RF_MODE_STB2

RF_MODE_STB3

RF_MODE_TX

RF_MODE_RX

Return Value

FAIL Operation failed

OK Operation successful

3.8 rf_get_mode

Function

```
uint8_t rf_get_mode(void)
```

Purpose

Read the current operation mode of the chip, including deepsleep/sleep/standby1/standby2/standby3/tx/rx

Return Value

Operation mode value.

3.9 rf_set_tx_mode

Function

```
RF_Err_t rf_set_tx_mode(uint8_t mode)
```

Purpose

Set the transmission mode, which can be set to single transmission or continuous transmission.

Parameter

Mode Transmit data mode

RF_TX_SINGLE

RF_TX_CONTINOUS

Return Value

FAIL Operation failed

OK Operation successful

3.10 rf_set_rx_mode

Function

```
RF_Err_t rf_set_rx_mode(uint8_t mode)
```

Purpose

Set the receiving mode, which can be set to single receiving, timeout receiving, continuous receiving.

Parameter

Mode Receive mode

RF_RX_SINGLE

RF_RX_SINGLE_TIMEOUT

RF_RX_CONTINUOUS

Return Value

FAIL Operation failed

OK Operation successful

3.11 rf_set_rx_single_timeout

Function

```
RF_Err_t rf_set_rx_single_timeout(uint16_t timeout)
```

Purpose

Set the timeout period for timeout reception. It is only valid in timeout reception mode.

Parameter

Timeout 1~65535 Timeout period, in ms

Return Value

FAIL Operation failed

OK Operation successful

3.12 rf_get_snr

Function

```
float rf_get_snr(void)
```

Purpose

Read the value of the signal-to-noise ratio SNR. The higher the SNR, the better the signal quality. This interface needs to be called when a data packet is received and before the RxDone interrupt is cleared. If the interrupt is cleared, this value will become invalid. The same is true for RSSI. The rf_irq_process() function provides a demonstration of the use of this function.

Example:

```
// RxDone IRQ received  
  
snr = rf_get_snr();  
  
rssi = rf_get_rssi();  
  
rf_receive(buf);
```

Return Value

SNR value

3.13 rf_get_rssi

Function

```
int8_t rf_get_rssi(void)
```

Purpose

Read the received signal strength value. This interface needs to be called when a data packet is received and before the RxDone interrupt is cleared. If the interrupt is cleared, this value will become invalid. The same is true for SNR. The rf_irq_process() function provides a usage demonstration of this function. For more usage, please refer to the RSSI application reference document.

Example:

```
// RxDone IRQ received  
  
snr = rf_get_snr();  
  
rssi = rf_get_rssi();  
  
rf_receive(buf);
```

Return Value

RSSI value

3.14 rf_get_channel_rssi

Function

```
int8_t rf_get_channel_rssi(void)
```

Purpose

PAN3029/3060 supports channel energy detection function, and the channel rssi value can be used to indicate the energy level of the channel. After the chip enters the RX state, this function can be called to obtain the channel detection energy and read the channel energy strength value.

Example:

```
//After the chip enters the RX state  
channel_rssi = rf_get_channel_rssi();
```

Return Value

RSSI value

3.15 rf_get_irq

Function

```
uint8_t rf_get_irq(void)
```

Purpose

Read interrupt register value. Used in rf_irq_process.

Return Value

Interrupt flags.

3.16 rf_clr_irq

Function

```
RF_Err_t rf_clr_irq(uint8_t flags)
```

Purpose

Clear the corresponding interrupt.

Parameter

Interrupt flags

REG_IRQ_MAPM_DONE


```
REG_IRQ_RX_PLHD_DONE
REG_IRQ_RX_DONE
REG_IRQ_CRC_ERR
REG_IRQ_RX_TIMEOUT
REG_IRQ_TX_DONE
```

Return Value

FAIL Operation failed
OK Operation successful

3.17 rf_set_refresh

Function

```
RF_Err_t rf_refresh(void)
```

Purpose

Register parameters are refreshed. Register values are not modified.

Return Value

FAIL Operation failed
OK Operation successful

3.18 rf_set_preamble

Function

```
RF_Err_t rf_set_preamble(uint16_t reg)
```

Purpose

Setting the chip to transmit the preamble is only valid for the transmitter and needs to be configured before transmitting data. When transmitting data, the transmission content consists of "preamble + data". The longer the preamble is, the longer the transmission time of the entire transmission content is.

Parameter

```
8~65535 Transmit preamble length
```

Return Value

FAIL Operation failed

OK Operation successful

3.19 rf_cad_on

Function

RF_Err_t rf_cad_on(uint8_t threshold, uint8_t chirps)

Purpose

Set the chip to turn on the CAD function. This function modifies the receiving threshold before the chip enters the receiving mode.

Parameters

Receiving threshold

CAD_DETECT_THRESHOLD_0A

CAD_DETECT_THRESHOLD_10

CAD_DETECT_THRESHOLD_15

CAD_DETECT_THRESHOLD_20

Number of chirps symbol detections

CAD_DETECT_NUMBER_1

CAD_DETECT_NUMBER_2

CAD_DETECT_NUMBER_3

Return Value

FAIL Operation failed

OK Operation successful

3.20 rf_cad_off

Function

RF_Err_t rf_cad_off(void)

Purpose

Set the chip to turn off the CAD function and restore the receiving threshold.

Return Value

FAIL Operation failed

OK Operation successful

3.21 rf_set_syncword

Function

```
RF_Err_t rf_set_syncword(uint8_t sync)
```

Purpose

Set the chip's synchronization word.

Parameter

Sync Synchronization word

Return Value

FAIL Operation failed

OK Operation successful

3.22 rf_get_syncword

Function

```
uint8_t rf_get_syncword(void)
```

Purpose

Read the chip's sync word.

Return Value

Sync word value

3.23 rf early interrupt interface

Functions

```
RF_Err_t rf_set_plhd(uint8_t addr, uint8_t len)
void rf_set_plhd_rx_off(void)
uint8_t rf_get_plhd_len(void)
RF_Err_t rf_plhd_receive(uint8_t *buf, uint8_t len)
```

Purpose

The early interrupt function is to check the decoded data during the chip reading a frame of data, determine whether it is what you want, and then decide whether to continue reading or abandon this frame of data. For more function descriptions, please refer to the "PAN3029_3060 Series Early Interrupt Application Reference Document".

3.24 rf_set_mapm_rx_on

Function

```
void rf_set_mapm_on(void)
```

Purpose

Enable mapm mode and mapm interrupt

Return Value

None

3.25 rf_set_mapm_rx_off

Function

```
void rf_set_mapm_off(void)
```

Purpose

Disable mapm mode and mapm interrupt

Return Value

None

3.26 rf_set_mapm_para

Function

```
void rf_set_mapm_para(stc_mapm_cfg_t *p_mapm_cfg)
```

Purpose

Set mapm mode related parameters

Parametres

p_mapm_cfg->fn is used to configure the number of fields in mapm preamble
p_mapm_cfg->fnm is used to configure the number of repetitions of each field in mapm preamble
p_mapm_cfg->gfs is used to configure the last group in the field, its ADDR position function selection
p_mapm_cfg->gn is used to configure the number of groups in a field
p_mapm_cfg->pgl is used to configure the number of preambles in the first group
p_mapm_cfg->pgn is used to configure the number of preambles in other groups
p_mapm_cfg->gn is the number of chirps before the syncword after all fields are sent

Return Value

None

3.27 rf_receive

Function

```
uint8_t rf_rcv_packet(uint8_t *buff)
```

Purpose

Receive a packet of data

Parameters

Buff - buffer used to receive data packets

Return Value

Length of received data packet

3.28 rf_enter_continuous_rx

Function

```
RF_Err_t rf_enter_continuous_rx(void)
```

Purpose

Set the chip to enter continuous receiving mode.

In this mode, the chip will always be in receiving state. When data is received, the chip will generate rx IRQ related interrupts and continue to receive.

Return Value

FAIL Operation failed

OK Operation successful

3.29 rf_enter_single_timeout_rx

Function

```
RF_Err_t rf_enter_single_timeout_rx(uint32_t timeout)
```

Purpose

Set the chip to enter the timeout receiving mode.

In this mode, the chip will receive once. After receiving the data, the chip will generate an rxdone IRQ interrupt and wait for the user's subsequent operation. When no data is received within the timeout period, the chip will generate an rxtimeout IRQ interrupt and wait for the user's subsequent operation. If you give up receiving, you need to actively exit the receiving state, otherwise the chip will start a timeout reception again. After use, the user software must first switch to STB3 to exit the receiving state before performing subsequent operations.

Parameter

Timeout

Return Value

FAIL Operation failed

OK Operation successful

3.30 rf_enter_single_rx

Function

```
RF_Err_t rf_enter_single_rx(void)
```

Purpose

Set the chip to enter the single receive mode.

In this mode, the chip will receive once. After receiving the data, the chip will generate an rx IRQ related interrupt and exit the

receive state. The user software needs to switch to STB3 to exit the receive state before performing subsequent operations.

Return Value

FAIL Operation failed

OK Operation successful

3.31 rf_single_tx_data

Function

```
RF_Err_t rf_single_tx_data(uint8_t *buf, uint8_t size, uint32_t *tx_time)
```

Purpose

Set the chip to enter single-shot transmission mode and transmit data.

In this mode, the chip will transmit once. When the transmission is completed, the chip will generate a txdone IRQ interrupt and exit the transmission state (the operating current is reduced). The user software needs to switch to STB3 to exit the transmission state before performing subsequent operations.

Parameters

Buf transmit buffer

Size packet length

Tx_time gets the transmission time of this data packet. The tx_time parameter is optional and can be omitted as needed.

Return Value

FAIL Operation failed

OK Operation successful

3.32 rf_enter_continuous_tx/rf_continuous_tx_send_data

Functions

```
RF_Err_t rf_enter_continuous_tx(void)
```

```
RF_Err_t rf_continuous_tx_send_data(uint8_t *buf, uint8_t size)
```

Purpose

Set the chip to enter the continuous transmission mode and transmit data.

In this mode, after the chip completes one transmission, the chip will generate a txdone IRQ interrupt and maintain the transmission state (the working current is still the transmission current, but no data is transmitted), and wait for the user's subsequent operation.

Afterwards, the user can choose to stop transmitting and switch the chip state to standby3; or choose to continue transmitting and directly call

rf_continuous_tx_send_data(). Continuing transmission does not require repeating the rf_enter_continuous_tx() operation.

Parameters

Buf transmit buffer

Size packet length

Return Value

FAIL Operation failed

OK Operation successful

3.33 rf_set_agc

Function

```
RF_Err_t rf_set_agc(bool NewState)
```


Purpose

Configure chip AGC. By default, AGC is configured and turned on when the chip is initialized.

Parameter

State AGC_OFF/ AGC_ON

Return Value

FAIL Operation failed

OK Operation successful

3.34 rf_set_dcdc_mode

Function

```
RF_Err_t rf_set_dcdc_mode(uint8_t dcdc_val)
```

Purpose

Set the chip DCDC mode, which is off by default. DCDC mode can obtain lower receiving current.

Before enabling this function, please contact and confirm whether the module supports this function, otherwise it may cause damage to the module.

Parameter

Dcdc_val mode

DCDC_ON

DCDC_OFF

Return Value

FAIL Operation failed

OK Operation successful

3.35 rf_set_ldr

Function

```
RF_Err_t rf_set_ldr(uint32_t mode)
```

Purpose

Set the chip LDR low-rate mode, which is off by default. The low-rate mode can further reduce the transmission data rate and obtain better transmission effect.

Parameter

Mode type

LDR_ON

LDR_OFF

Return Value

FAIL Operation failed

OK Operation successful

3.36 rf intelligent search interface

Functions

```
RF_Err_t rf_set_auto_sf_tx_preamble(int sf, int sf_range[], int size, int chirp_counts[])  
RF_Err_t rf_set_auto_sf_rx_on(int sf_range[], int size)  
RF_Err_t rf_set_auto_sf_rx_off(void)  
int calculate_chirp_count(int sf_range[], int size, int chirp_counts[])
```

Purpose

The intelligent search function can realize intelligent identification of SF parameters in the channel during reception, so as to achieve the purpose of receiving different SF signal data. For more function descriptions, please refer to "PAN3029_3060 Series_Intelligent Search Application Reference Document".

3.37 get_chirp_time

Function

```
uint32_t rf_get_chirp_time(uint8_t bw, uint8_t sf)
```

Purpose

Calculate the duration of a single chirp.

Parameters

SF: SF_5/SF_6/SF_7/SF_8/SF_9/SF_10/SF_11/SF_12

BW: BW_62_5K/BW_125K/BW_250K/BW_500K

Return Value

Single chirp duration, unit: us.

3.38 check_cad_rx_inactive

Function

```
bool check_cad_rx_inactive(uint32_t one_chirp_time)
```

Purpose

Use CAD to detect the current channel status, used for channel detection before CAD reception.

Parameter

one_chirp_time: duration of a single chirp

Return Value

LEVEL_ACTIVE The channel is active and should continue to receive

LEVEL_INACTIVE The channel has no signal and should stop receiving.

4 SDK Porting

This chapter describes the SDK migration process and precautions in detail. If in Keil configured floating point hardware to "Not Used", unchecked "One ELF Section per Function", and set the optimization level to "default", after compiling the code, the Flash size required by the SDK is 9125 (8.91kB) and the SRAM size required is 1485 (1.45kB).

The SDK "pan_port.c" provides the interface functions that users need to port. To run the SDK correctly, the following interfaces need to be implemented:

- spi_cs_set_high(), pull high SPI chip select.
- spi_cs_set_low(), pull low SPI chip select.
- spi_readwritebyte(), SPI data transfer.
- rf_delay_ms(), rf_delay_us(), delay function.
- RX_IO/TX_IO Implement the chip's IO control interface, including rf_antenna_init, rf_antenna_tx rf_antenna_rx, rf_antenna_close.
RX_IO/TX_IO is implemented in the SDK through SPI read and write registers, and RX_IO/TX_IO is implemented through SPI control of the chip's internal GPIO4 and GPIO3. Users can also use external MCU IO to control RX_IO/TX_IO. The specific implementation depends on the board design.
- TCXO_IO implements the chip active crystal oscillator IO control interface, including rf_tcxo_init and rf_tcxo_close.
In SDK, this part is implemented by reading and writing SPI registers, and TCXO_IO is implemented by controlling GPIO5 inside the chip through SPI.
Users can also use external MCU IO to control TCXO_IO. The specific implementation depends on the PAN3029/3060 series module version and MCU board design.

5 PAN3029/3060 Series Status Diagram

This section describes the state switching diagram of the PAN3029/3060 series chips, which is convenient for users to have a deeper understanding of the working principle of the chip. Among them, the wake-up process from DeepSleep to STB3 (standby3) and the configuration initialization correspond to the `rf_init` function in the SDK; STB3 to DeepSleep corresponds to the `rf_deepsleep` function in the SDK. The switching process from Sleep to STB3 corresponds to the `rf_sleep_wakeup` function in the SDK; STB3 to Sleep corresponds to the `rf_sleep` function in the SDK.

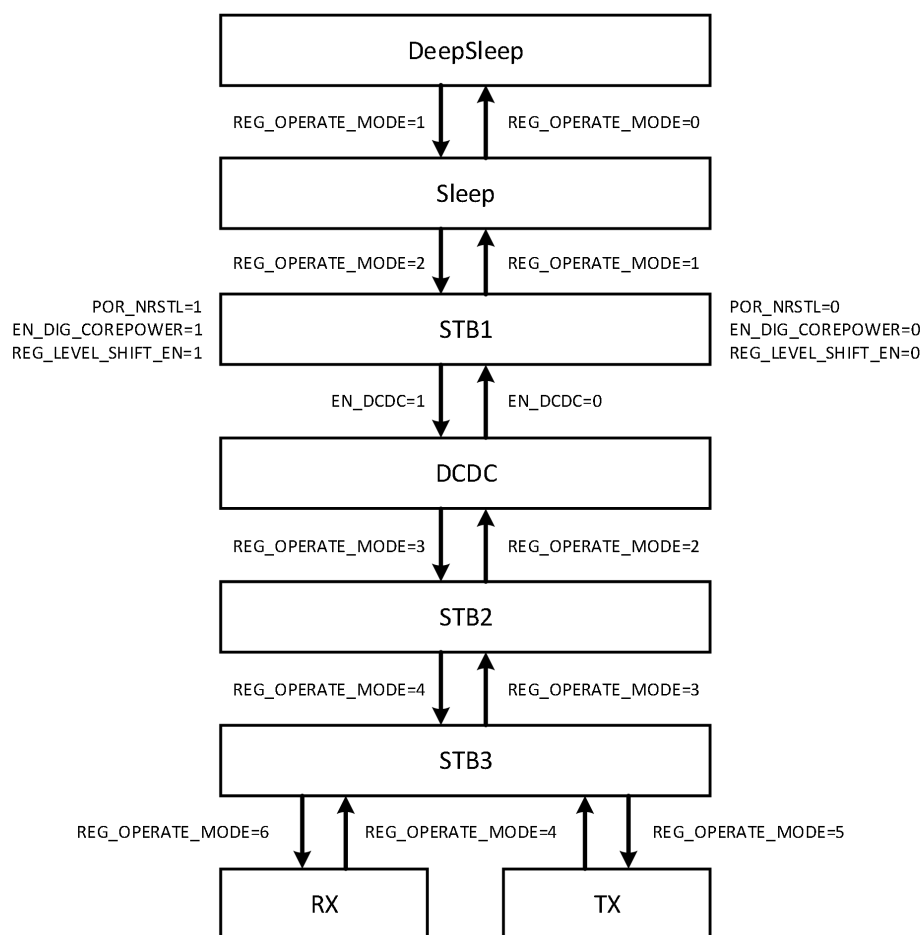


Figure 6-1 PAN3029 power-on process

The PAN3029/3060 series chips have seven states after power-on, including Deep Sleep mode (DeepSleep), Sleep mode (Sleep),

LDO working mode (STB1), OSC working mode (STB2), OSC output mode (STB3), TX mode and RX mode.

All states can be switched through the configuration register.

Note: If the chip peripheral circuit does not support DCDC, you can directly enter STB2 from STB1 without configuring EN_DCDC. SDK does not use DCDC function by default. If you need to use DCDC function, please refer to the corresponding application document.

6 Application Notes

6.1 Low Power Sleep

Regarding low-power sleep, the SDK provides four interface functions.

One is the Sleep mode. It includes the `rf_sleep` function from STB3 to Sleep and the `rf_sleep_wakeup` function from Sleep to STB3.

One is the DeepSleep mode. It includes the `rf_deepsleep` function from STB3 to DeepSleep and the `rf_init` function from DeepSleep to STB3. Note that after the PAN3029/3060 series chip enters deep sleep and wakes up again, the original configuration parameters need to be reconfigured, and the transceiver mode also needs to be reconfigured.

6.2 CAD Function

The CAD function modifies the chip's receiving threshold, which may affect the chip's receiving sensitivity.

After use, you need to turn off the CAD function.

For more function descriptions, please refer to "PAN3029_3060 Series_CAD Application Reference Document".

6.3 Sensitivity and RSSI

Reading the signal strength value needs to be called when receiving a data packet and before clearing the `rxdone` interrupt. If the interrupt is cleared, this value will become invalid. The RSSI measurement range is -12 to -110, and the measurement range is slightly different in different parameter (SF, BW) modes.

When you need to read the chip RSSI, it is recommended to turn on AGC to improve the chip RSSI accuracy. For more function descriptions, please refer to the "PAN3029_3060 Series_RSSI Application Reference Document".

6.4 DCDC Function

When the DCDC function is turned on, the chip's receiving current can be reduced to 4.77mA. This function is turned off by default. The SDK provides an interface function to turn the DCDC function on or off.

```
RF_Err_t rf_set_dcdc_mode(uint8_t dcdc_val)
```

Parameter dcdc_val

DCDC_ON On

DCDC_OFF Off

The DCDC function can be enabled in STB1~STB3 and RX mode.

After enabling the DCDC function, when the chip needs to send, the DCDC function needs to be manually disabled in advance.

Before enabling the DCDC function, please contact and confirm whether the module supports this function, otherwise it may cause damage to the module.

6.5 EFUSE Function

PAN3029/3060 series chips support 82Bytes of eFuse space area for customers to use, which can be used to store data that will not be lost when power is off. Regarding the use of eFuse function, the SDK provides four interface functions.

Function

```
uint32_t rf_efuse_on(void)
```

Purpose

Enable eFuse function

Parameters

none

Return Value

FAIL Operation failed

OK Operation successful

Function

```
uint32_t rf_efuse_off(void)
```

Purpose

Disable the eFuse function

Parameters

none

Return Value

FAIL Operation failed

OK Operation successful

Function

```
uint8_t rf_efuse_read_byte(uint8_t reg_addr, uint8_t efuse_addr)
```

Purpose

Read the data stored at the specified efuse address

Parameters

reg_addr efuse entry address, client area defaults to 0x3c

efuse_addr efuse internal address, client area 0x2d~0x7f

Return Value

value Read the data returned by the specified efuse address

Function

```
void rf_efuse_write_byte(uint8_t reg_addr, uint8_t efuse_addr, uint8_t value)
```

Purpose

Write the data you want to store to the specified efuse address

Parameters

reg_addr efuse entry address, client area defaults to 0x3c

efuse_addr efuse internal address, client area 0x2d~0x7f

value The data to be written to efuse

Return Value

None

Special attention should be paid to eFuse reading and writing:

1. eFuse can only be programmed once, and once programmed, it cannot be modified.
2. The same address of eFuse cannot be written repeatedly, otherwise it is easy to damage the eFuse device.
3. The voltage range of writing eFuse is 2.25V to 2.75V, and the typical value is 2.5V. Therefore, when configuring eFuse, it is recommended to use an external VDD1 2.5V voltage.

The read voltage can normally use 3V voltage.

4. eFuse only supports read and write operations in STB3, do not read and write in other modes.
5. eFuse area does not support continuous read and write operations of SPI and I2C.
6. The maximum clock rate of eFuse read and write is 8MHz.

6.6 RF Parameters Description

Channel Bandwidth (BW)

The channel bandwidth is the lower and upper frequency limits of the signal that is allowed to pass through the channel. It can be understood as a frequency passband, and its value range is 62.5KHz, 125KHz, 250KHz, and 500KHz. For example, if the passband allowed by a channel is 1.5kHz to 15kHz, then its bandwidth is 13.5kHz. Increasing BW can increase the effective data rate to shorten the transmission time, but at the expense of some receiving sensitivity, thus affecting the communication distance.

Spreading Factor (SF)

The SF value range is SF5-12. The smaller the value, the higher the transmission rate and the shorter the transmission distance. On the contrary, the larger the SF value, the slower the transmission rate and the longer the transmission distance. When the same amount of data is transmitted, the larger the SF value, the longer the transmission time.

Coding Rate (CR)

CR is the ratio of the useful part (non-redundant) in the data stream, and the value range is 1-4, corresponding to $1=4/5$, $2=4/6$, $3=4/7$, $4=4/8$.

That is, if the coding rate is k/n , for every k bits of useful information, the encoder generates a total of n bits of data, of which $n-k$ are redundant. The larger the CR, the more redundant data is transmitted in a single transmission, the lower the effective data rate, and the longer the transmission time.

Checksum (CRC)

CRC is set on the TX side, but not on the RX side. The RX side receives the information in the signal and reads the CRC in the frame structure to decide whether to perform a CRC check.

Low rate mode

The value range is 1 or 0. When it is 1, the low-rate mode is turned on. The default is off. After turning it on, the communication rate will be reduced, the receiving sensitivity will increase by 1-2dBm, and the communication distance will be slightly increased.

Number of preamble symbols

The value range is 8-65535, and the default value is 8. When implementing the air wake-up application scenario, the number of preamble symbols can be increased, and the transmission time will also be increased, so that the receiving end can be awakened by a long preamble in low power mode.

Note: The RF parameters above need to be selected and configured according to the actual application scenario, and the RF configuration of the transmitter and receiver must be consistent for normal communication. For the selection of RF data transmission time and parameters, please refer to the calculation tool in the SDK.

6.7 4-wire SPI and FIFO

External devices can access the registers and FIFOs in the chip through the four-wire SPI mode.

The PAN3029/3060 series chips implement the SPI bus slave, which is used to read and write registers and FIFO. The SPI bus is a four-wire system, which are:

- SCK (clock)
- CSN (chip select signal, low level active)
- MOSI (data input)
- MISO (data output)

Among them, SCK, CSN, and MOSI are controlled by the host Master, and MISO is controlled by the Slave.

During the communication process, the transmission starts when the CSN level is pulled low, and ends when the CSN level is pulled high. The host Master sends data through MOSI and receives data through MISO. Data is generated when SCK falls, and data is sampled when it rises. The information transmitted by the Master consists of two parts: Address Byte and Data Byte. Among them, the first 7 bits of Address Byte are the address bit *addr*; the last 1 bit is the read-write bit *wr*, which is set to 1 during write operation and set to 0 during read operation.

SPI has three transfer modes:

- Single: Single byte transmission mode. The information is only 2 bytes, and the Master sends the Address Byte through MOSI. If it is a write operation, the Master continues to send the Data Byte through MOSI; if it is a read operation, the Master reads the Data Byte replied by the Slave on MISO.
- Burst: Burst continuous transmission mode. When the information is larger than 2 bytes, the Address Byte is followed by several Data Bytes. There is no need to add Address Bytes between Data Bytes. The Slave will automatically increment the address between each Data Byte. The CSN signal is pulled high after the last Data Byte, and the rest of the information transmission process is maintained at a low level.
- FIFO: FIFO read/write mode. In this mode, single byte or continuous transmission can be realized. The transmission rules are the same as those in Single mode and Burst mode. The difference is that the address bit *addr* in the Address Byte can only be configured as 7'h1, and the Slave does not perform address increment operations between Data Bytes.

The SPI write timing is as follows:

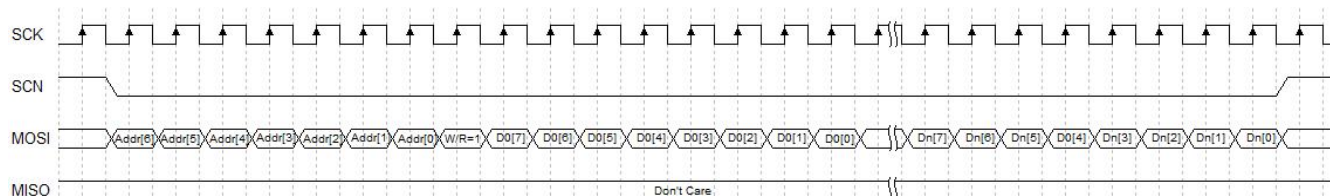


Figure 7-1 SPI write timing

The SPI read timing is as follows:

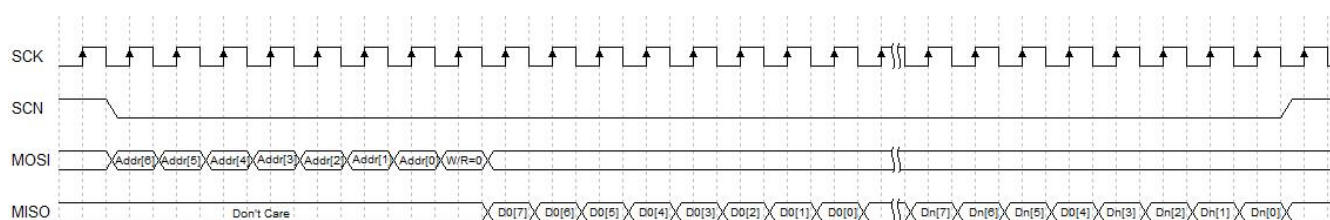


Figure 7-2 SPI read timing

The PAN3029/3060 series chips have a 255-byte FIFO to store the TX module transmission data and the RX module decoding data.

FIFO is composed of a single-port RAM and can only store and read a single packet of data information. If there is already a packet of data in the FIFO, this packet of data should be read before writing, otherwise the previous packet of data in the FIFO will be overwritten.

7 Appendix

7.1 Transmitting power table

档位	频率	410MHz		430MHz		450MHz		460MHz	
		功率(dBm)	电流(mA)	功率(dBm)	电流(mA)	功率(dBm)	电流(mA)	功率(dBm)	电流(mA)
1		-18.733	7.267	-18.165	7.418	-18.767	7.546	-19.679	7.607
2		-8.276	9.42	-7.936	9.563	-8.596	9.671	-9.548	9.758
3		1.599	17.935	1.748	18.349	0.947	18.725	-0.111	18.932
4		3.925	20.301	4.513	20.661	4.114	20.967	3.348	21.439
5		4.921	21.76	5.329	22.183	4.79	22.662	3.865	23.076
6		5.413	22.933	5.501	23.385	4.776	23.811	3.841	24.136
7		7.104	26.605	7.611	26.904	7.345	27.468	6.805	28.54
8		7.655	28.062	8.152	28.282	7.951	28.995	7.491	30.298
9		8.638	30.074	9.185	30.663	8.755	31.328	8.117	32.387
10		8.966	32.23	9.29	31.92	9.106	32.679	8.897	34.62
11		10.274	35.676	10.746	36.083	10.571	37.273	10.298	39.578
12		10.816	39.495	10.988	38.391	10.753	38.94	10.56	41.211
13		11.567	41.972	11.815	41.256	11.612	42.165	11.468	44.97
14		12.83	46.616	13.205	46.748	13.077	48.484	12.919	52.149
15		13.782	50.643	14.252	51.595	14.182	54.016	14.001	58.272
16		14.871	55.72	15.404	57.538	15.36	60.671	15.015	64.89
17		15.404	58.355	15.957	60.647	15.797	63.599	15.317	67.146
18		16.052	63.254	16.602	65.475	16.522	69.186	16.267	74.527
19		16.493	66.892	16.956	68.616	16.899	72.499	16.738	78.608
20		17.414	73.954	17.868	75.84	17.8	80.261	17.632	86.882
21		18.229	87.976	18.297	85.286	17.994	86.519	17.694	90.391
22		19.436	102.128	19.414	98.008	19.07	98.768	18.736	102.895
23		19.754	105.378	19.771	101.687	19.433	102.835	19.105	107.278

Note: The power level is a parameter of the rf_set_tx_power function.
The transmit power and transmit current of other frequency bands will be slightly different.